

ADVANCES IN AI
PROF. DR. DENNIS MÜLLER

AI Age Transformation

Nick Grabowski, 912350



https://github.com/nickno7/age_transformation

16.02.2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Projektbeschreibung	1
1.2	Motivation	1
1.3	Relevanz	1
2	State of the Art	2
3	Methodik	2
3.1	StyleGAN-Implementierung	3
3.1.1	Einleitung	3
3.1.2	Custom Layers	3
3.1.3	Mapping Network	3
3.1.4	Synthesis Network	4
3.1.5	Truncation Trick	4
3.1.6	Fazit	4
3.2	Encoder	4
3.2.1	Datensatz	5
3.2.2	Architektur	6
3.3	Latent Optimizer	7
3.3.1	Ablauf	8
3.4	Age Transformation	9
3.4.1	Interpolation	10
4	Ergebnisse	10
5	Herausforderungen	11
6	Next Steps	12
	Anhang	14

1 Einleitung

1.1 Projektbeschreibung

In diesem Projekt widme ich mich der KI-basierten Alterstransformation mithilfe des StyleGANs. Dabei geht es sowohl um die Gesichtsalterung als auch -verjüngung. Für eine realistische Alterstransformation ist es wichtig die Altersmerkmale zu verändern, ohne die Identität der Person zu verlieren. Mithilfe des interpretierbaren latenten Raum der StyleGAN Architektur, kann man Gesichtsmerkmale, wie das Alter, anpassen indem man eine latenten Repräsentation der Person mit einem Vektor addiert, der das Alter verändert. Basierend auf der Richtung des Vektors kann die Person entweder gealtert oder verjüngert werden.

Das Prokekt umfasst die Möglichkeit, mithilfe eines vortrainierten StyleGANs realistische Gesichter zu erzeugen und das Alter dieser Gesichter zu transformieren. Dabei kommt es zum Einsatz eines Encoders, der gelernt hat anhand von Input-Bildern latente Vektoren des StyleGAN Raums zu erzeugen, um auch mit echten Personen arbeiten zu können und deren Alter anzupassen.

1.2 Motivation

Ich habe mich für dieses Projekt entschieden, weil ich es sehr spannend finde, wie schnell sich KI im Hinblick auf die realistische Bildgenerierung entwickelt und verbessert hat. Vor allem this-person-does-not-exist.com [1] hat mich und auch viele andere Menschen fasziniert, da man gesehen hat zu was GANs und vor allem das StyleGAN im Stande ist. Ausserdem war es mir wichtig ein visuelles Projekt durchzuführen, bei dem man direkt die Ergebnisse sehen kann.

Die Möglichkeit das erste mal mit generativen Modellen zu arbeiten und ein tieferes Verständnis dafür zu bekommen, wie sie funktionieren hat mich zusätzlich gereizt.

1.3 Relevanz

Es stellt sich die Frage, wofür man sowas überhaupt braucht und was es für Anwendungsfälle gibt. Die KI-basierte Alterstransformation ist in vielen Bereichen relevant und hilfreich. Vor allem in der Forensik ist es eine geeignete Methode, um langjährig vermisste Personen zu finden, indem man Bilder erstellt, die zeigen wie die Person heute aussehen könnte. Speziell bei Kindern ist das hilfreich, da sich die Gesichtsstrukturen und das Aussehen in wenigen Jahren stark verändern.

Ein weiterer Anwendungsfall ist die Entertainment-Industrie. Alterstransformationen werden genutzt, um realistische vergangene oder zukünftige Versionen von Schauspielern in Szene zu setzen, ohne andere Schauspieler dafür verwenden zu müssen.

Zudem können derweilige Alterstransformationen für die Rekonstruktion historischer Personen verwendet werden, um zu zeigen, wie verstorbene Personen aus der Vergangenheit heute aussehen würden.

2 State of the Art

Die Alterung von Gesichtern mit Künstlicher Intelligenz hat in den letzten Jahren erhebliche Fortschritte gemacht. Ein zentraler Punkt ist die Nutzung von Generative Adversarial Networks (GANs), die es ermöglichen, realistische Bilder zu erzeugen und zu manipulieren [2].

Ein bedeutender Meilenstein war die Entwicklung des StyleGAN, das eine hohe Qualität und Kontrolle bei der Bildsynthese bietet [3]. Untersuchungen haben gezeigt, dass der latente Raum von GANs semantische Informationen enthält, die gezielte Modifikationen von Attributen wie Alter, Geschlecht oder Gesichtsausdruck ermöglichen

Einige Jahre nach der Veröffentlichung von GANs wurde der latente Raum und dessen Interpretierbarkeit erforscht. Dabei wurde untersucht, wie verschiedene Attribute, wie Alter, Geschlecht oder Gesichtsausdruck, im latenten Raum von GANs repräsentiert sind. Es wurde festgestellt, dass durch gezielte Modifikation der latenten Vektoren spezifische Merkmale eines Gesichts verändert werden konnten, was eine präzise Steuerung des Alters ermöglicht [4].

Eine weitere Methode zur Gesichtsveränderung ist CycleGAN, das unüberwacht zwischen zwei Domänen übersetzen kann, beispielsweise zwischen jungen und alten Gesichtern. Dies ermöglicht die Alterung oder Verjüngung von Gesichtern ohne paarweise Trainingsdaten [5]. Die Invertierbarkeit von GANs ist ein weiteres relevantes Forschungsgebiet. Es untersucht, wie man von einem gegebenen Bild auf den entsprechenden latenten Code schließen kann, um gezielte Bildmanipulationen durchzuführen. Studien haben gezeigt, dass es möglich ist, den latenten Code effizient zu rekonstruieren [6].

Trotz dieser Fortschritte gibt es weiterhin Herausforderungen, wie das Problem des Mode Collapse, bei dem bestimmte Merkmale oder Objekte nicht korrekt generiert werden. Untersuchungen haben gezeigt, dass GANs dazu neigen, bestimmte Objektklassen zu vernachlässigen, was die Notwendigkeit weiterer Forschung zur Verbesserung der Modellgeneralisierung unterstreicht [7].

3 Methodik

Das Projekt ist in verschiedene Kapitel unterteilt, da viele Bausteine für die Umsetzung des Ziels erforderlich waren. Ich habe mit der PyTorch-Umgebung gearbeitet, um sowohl die StyleGAN-Architektur, als auch den Encoder sowie den Latent Optimizer zu programmieren. Aus zeitlichen Gründen habe ich für das StyleGAN ein vortrainierte Netz verwendet. Dadurch hatte ich die Möglichkeit mit zufälligen z Vektoren künstliche Gesichter zu erzeugen und für den weiteren Verlauf zu nutzen, z.B. für das Training des Encoders.

3.1 StyleGAN-Implementierung

3.1.1 Einleitung

Diese Implementierung basiert auf der StyleGAN-Architektur, die eine progressive Wachstumsstrategie für Generator-Netzwerke mit stilbasierten Steuerungen kombiniert. Die Architektur umfasst verschiedene essentielle Bausteine, darunter:

3.1.2 Custom Layers

- **MyLinear**: Ein Fully Connected Layer mit gleichmäßigem Lernraten-Skalieren.
- **MyConv2d**: Eine Faltungsschicht mit Upscaling-Option.
- **NoiseLayer**: Fügt Rauschen für Variation in den Features hinzu.
- **StyleMod**: Stilmodulations-Schicht zur bedingten Skalierung und Verschiebung von Features.
- **PixelNormLayer**: Normierung der Features auf Pixelebene.
- **BlurLayer**: Implementierung eines Tiefpassfilters zur Stabilisierung des Trainings.
- **Upscale2d**: Hochskalierungsfunktion für Bilder.

3.1.3 Mapping Network

Das Mapping-Netzwerk ($G_{mapping}$) verarbeitet den latenten Raum z durch mehrere vollständig verbundene Schichten und projiziert ihn in den **W-Raum**, um feinkörnige Kontrolle über das generierte Bild zu ermöglichen.

3.1.4 Synthesis Network

Das $G_{\text{synthesis}}$ -Netzwerk wandelt den latenten Vektor w schrittweise in ein hochauflösendes Bild um. Es besteht aus:

- **InputBlock**: Initialisiert das Feature-Grid (4×4 Pixel) entweder mit einer gelernten Konstante oder durch Transformation des latenten Codes.
- **GSynthesisBlock**: Fortlaufende Blöcke mit Upscaling und Konvolutionen.

Das Netzwerk wächst progressiv von niedriger zu hoher Auflösung und erzeugt so feine Details in den Bildern.

3.1.5 Truncation Trick

Eine optionale **Truncation-Schicht** interpoliert den latenten Vektor in Richtung des Durchschnittsvektors, um die Diversität der generierten Bilder zu kontrollieren.

3.1.6 Fazit

Diese Implementierung bildet eine solide Grundlage für StyleGAN-Modelle und kann durch Techniken wie progressive Wachstumsstrategien und optimierte Normalisierungsansätze weiter verbessert werden.

3.2 Encoder

Das folgende Modul implementiert ein Encoder-Netzwerk, das Bilder in den latenten Stilraum ($W+$) eines vortrainierten StyleGAN-Modells projiziert. Zudem werden Funktionen zur Generierung eines synthetischen Datensatzes sowie zum Training und zur Evaluierung des Encoders bereitgestellt.

Ich führe die Alterstransformation mithilfe einer Manipulation des latenten Vektors des StyleGANs durch. Deswegen ist der Encoder für die Aufgabe, ein Input Bild, das nicht vom StyleGAN generiert wurde, in den latenten Raum des StyleGANs zu projizieren, um den Vektor anschließend manipulieren und das Alter verändern zu können.

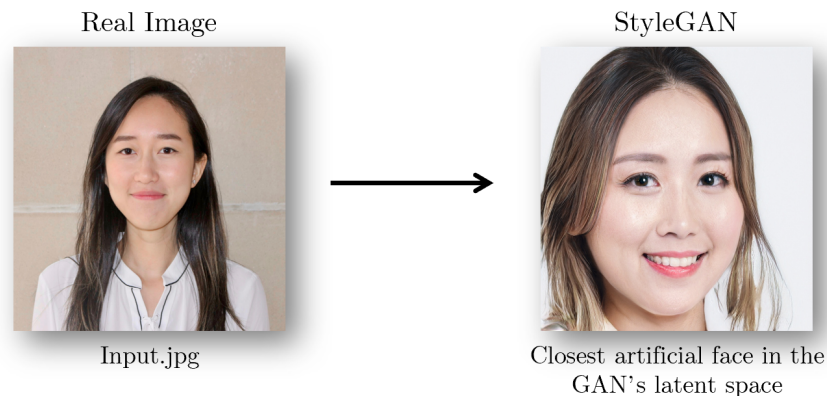


Abbildung 1 Quelle: <https://github.com/sshh12/Inverse-Style-GAN>

3.2.1 Datensatz

Für den Encoder habe ich mithilfe der StyleGAN-Implementierung einen eigenen Datensatz mit 50.000 Bild, Latent Vektor Paaren erstellt. Zur Generierung des Datensatzes wurden folgende Schritte durchgeführt:

1. Zufälliges Generieren von z-Vektoren mit 512 Dimensionen.
2. z-Vektor geht durch das Mapping-Netzwerk des StyleGAN, um den latenten w-Vektor zu erhalten, in dem wertvolle Informationen, wie die Altersstruktur stecken.
3. Erzeugung der zugehörigen Bilder mit dem StyleGAN-Synthesemodell.
4. Speicherung der Bilder und der zugehörigen latenten Vektoren.

Dieser Datensatz war notwendig, um ein Encoder-Netzwerk zu trainieren, das lernt anhand der Gesichter (Input), dazugehörige latente Vektoren zu generieren, die das Gesicht möglichst genau repräsentieren.

```
def generate_dataset(device, g_mapping, g_synthesis, num_samples=50000):  
    for i in range(num_samples):  
        z_sample = torch.randn(1, 512).to(device) # Zufälliger z-Vektor  
        latent = g_mapping(z_sample) # Mapping zu w-Vektor  
        with torch.no_grad():  
            image = g_synthesis(latent) # Gesicht generieren  
            save_image(image.cpu(), f"images/{i:05d}.png")  
            torch.save(latent.cpu(), f"latents/{i:05d}.pt")
```

„Gekürzte/Vereinfachte Darstellung“

3.2.2 Architektur

Der Encoder basiert auf einem vortrainierten ResNet-50-Modell, das zur Extraktion tieferer Bildmerkmale verwendet wird. Anschließend wird eine Abbildung auf den latenten Stilraum von StyleGAN durchgeführt. Die Architektur umfasst:

- Eine modifizierte ResNet-50-Architektur zur Feature-Extraktion.
- Eine zusätzliche Convolution-Schicht zur Reduktion der Merkmalsdimension.
- Fully Connected Layers zur Projektion der Bildmerkmale in den latenten Raum.

```
class Encoder(nn.Module):
    def __init__(self, latent_dim=512, w_plus_layers=18):
        super(Encoder, self).__init__()
        self.resnet = resnet50(pretrained=True)
        self.resnet = nn.Sequential(*list(self.resnet.children())[:-2])
        self.conv = nn.Sequential(
            nn.Conv2d(2048, latent_dim, kernel_size=1, stride=1),
            nn.BatchNorm2d(latent_dim),
            nn.LeakyReLU(0.2)
        )
        self.flatten = nn.Flatten()
        self.fc1 = nn.Sequential(
            nn.Linear(latent_dim * 7 * 7, 1024),
            nn.Dropout(0.5),
            nn.LeakyReLU(0.2)
        )
        self.fc2 = nn.Linear(1024, latent_dim * w_plus_layers)

    def forward(self, x):
        x = self.resnet(x)
        x = self.conv(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = x.view(-1, 18, 512)
        return x
```


Trainingsprozess

Das Encoder-Modell wird mit einem generierten Datensatz trainiert. Der Trainingsprozess umfasst:

- Verwendung einer Kombination aus Datensatztransformationen, wie Größenänderung, zufällige Zuschnitte und Normalisierung.
- Nutzung der `logcosh`-Verlustfunktion zur Messung der Differenz zwischen den vorhergesagten und echten latenten Vektoren.
- Anwendung von Adam als Optimierungsalgorithmus mit einem Learning-Rate-Scheduler.
- Speicherung und Laden von Checkpoints, um das Training bei Unterbrechungen fortzusetzen.

```
optimizer = optim.Adam(encoder.parameters(), lr=0.1, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
                                                         mode='min', factor=0.5, patience=2, min_lr=1e-6)

for epoch in range(start_epoch, epochs):
    encoder.train()
    epoch_loss = 0
    for imgs, true_latents in train_loader:
        imgs, true_latents = imgs.to(device), true_latents.to(device)
        predicted_latents = encoder(imgs)
        loss = logcosh_loss(true_latents, predicted_latents)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Fazit

Dieses Modul stellt ein vollständiges Pipeline-System für die Encodierung von Bildern in den latenten Stilraum von StyleGAN bereit. Es umfasst sowohl die Datensatzerstellung als auch die Trainings- und Evaluierungsprozesse des Encoders.

3.3 Latent Optimizer

Da man durch den Encoder nur eine Annäherung an das echte Inputbild erhält und der Output diesem meist noch nicht ähnlich genug ist, war eine weitere Methode notwendig. Dafür habe ich den Latent Optimizer implementiert. Dieser dient dazu ein möglichst präzises

latent Embedding zu erhalten, dass das Input Bild so genau wie möglich rekonstruiert. Das ist vor allem wichtig, um eine realistische Alterstransformation durchführen zu können, ohne die Identität zu verlieren.

Um die Differenz zwischen dem rekonstruierten Bild (Output des Encoders) und dem realen Bild zu minimieren, wird ein Optimierungsprozess eingesetzt, um den latenten Vektor schrittweise zu verfeinern.

3.3.1 Ablauf

Der Prozess nimmt den latenten Vektor, den der Encoder generiert, als Startpunkt zur Optimierung. Ein ResNet-Modell wird eingesetzt, um die perzeptuellen Merkmale des Bilds zu extrahieren und zwischen realem und generiertem Bild vergleichen zu können.

Außerdem wird der MSE-Loss eingesetzt, um den pixelweisen Unterschied zwischen den beiden Bildern zu verringern.

Der gesamte Loss setzt sich aus einer Summe des Perceptual- und MSE-Loss zusammen, sodass man folgende Loss-Funktion erhält.

$$\text{loss} = 0.6 \cdot \text{perceptual_loss} + 0.4 \cdot \text{mse}$$

```
def optimize_latent(image, device, encoder, g_synthesis):
    """Optimizes latent representation of an input image."""
    upsample = torch.nn.Upsample(scale_factor = 224/1024,
                                  mode = 'bilinear')

    img_p = upsample(image.clone())
    perceptual = ResNet_perceptual().to(device)

    # MSE loss object
    mse_loss = nn.MSELoss()

    # send image through encoder for first approximation
    latent = encoder(image)
    # Optimize latent code in each backward step
    optimizer = optim.Adam([latent], lr=0.01, betas=(0.9, 0.999), eps=1e-8)

    loss_ = []

    # latent optimization loop
    for i in range(2000):
        optimizer.zero_grad()
        syn_img = g_synthesis(latent)
```

```
syn_img = (syn_img + 1.0)/2.0

# calculate losses
mse, per_loss = loss_function(syn_img, image, img_p, mse_loss,
                              upsample, perceptual)

loss = 0.6 * per_loss + 0.4 * mse
loss.backward()
optimizer.step()

loss_np = loss.detach().cpu().numpy()
loss_mse = mse.detach().cpu().numpy()
loss_per = per_loss.detach().cpu().numpy()
loss_.append(loss_np)

# print loss and save image every 500th iteration
if (i + 1) % 500 == 0:
    print(f"Iteration{i+1}: loss -- {loss_np},
          mse_loss -- {loss_mse},  percep_loss -- {loss_per}")
    save_image(syn_img.clamp(0,1),
              f"outputs/face_{i+1}_iterations.png")

return latent
```

3.4 Age Transformation

Der latente Raum des StyleGANs ist interpretierbar und in den latenten Vektoren sind viele Informationen über Gesichtsmerkmale enthalten [4].

Diese Eigenschaft lässt sich ausnutzen, um Gesichter auf verschiedenste Weisen zu manipulieren und transformieren. Unter anderem kann man durch gezielte Manipulation des latenten Vektors auch das Alter anpassen.

Durch lineare Trennverfahren konnte identifiziert werden, welche Richtungen es im latenten Raum gibt, die mit bestimmten Gesichtsmerkmalen korrelieren.

Diese Richtungen/Boundaries habe ich mir zu Nutzen gemacht, indem ich die Boundary, die das Alter der Person im latenten Raum beschreibt für die Alterstransformation nutze. Die Alterstransformation erfolgt dann durch Verschiebung des latenten Vektors in Richtung der Boundary. Abhängig davon, in welche Richtung man den latenten Vektor verschiebt wird das Gesicht entweder verjüngert oder gealtert.

Der neue latente Vektor wird dementsprechend wie folgt berechnet:

$$\mathbf{w}_{\text{aged}} = \mathbf{w} + \alpha \cdot \mathbf{b}$$

Dabei bestimmt der Faktor Alpha α wie stark die Alterung sein soll.

Wählt man einen negativen Wert, wird das Gesicht verjüngert. Bei einem positiven Wert hingegen wird das Gesicht gealtert.

Diesen neuen latenten Vektor $\mathbf{w}_{\text{aged}}$ kann man anschliessend in das $G_{\text{synthesis}}$ -Netzwerk geben, um das entsprechend gealterte Bild zu erhalten.

3.4.1 Interpolation

Um den Alterungsprozess anschaulich darzustellen, wurde eine Interpolation zwischen dem ursprünglichen latenten Vektor w und dem gealterten Vektor $\mathbf{w}_{\text{aged}}$ durchgeführt. Dabei wird schrittweise ein neuer latenter Vektor berechnet:

$$w_{\text{interp}} = (1 - a) \cdot w + a \cdot w_{\text{aged}}$$

mit α als Interpolationsparameter im Bereich $[0,1]$.

Dieser Interpolationsprozess ermöglicht eine fließende Übergangsanimation zwischen dem ursprünglichen und dem gealterten Gesicht. In jedem Schritt wird ein Bild aus dem interpolierten latenten Vektor mit dem StyleGAN-Synthesemodell generiert und gespeichert. Die einzelnen Bilder werden dann zu einer GIF-Animation zusammengefügt, um die Alterung visuell darzustellen.

```
num_steps = 75

# Generate interpolated images
interpolated_images = []
with torch.no_grad():
    for a in np.linspace(0, 1, num_steps):
        z = ((1 - a) * latent) + (a * aged_w)
        result = g_synthesis(z)
        result = (result.clamp(-1, 1) + 1) / 2.0 # Normalize
        result = result.cpu()
        interpolated_images.append(result)
```

4 Ergebnisse

Die Alterstransformation hat sehr gut mit generierten Gesichtern funktioniert und äußerst realistische Alterungen simuliert. Die Ergebnisse sind in der README des GitHub Repositories zu finden. Es ist zu erkennen, dass sowohl die Alterung als auch die Verjüngung funktioniert und gute Transformationen hervorbringen.

Die Alterstransformation mit realen Gesichtern hat schlechtere Ergebnisse erzielt. Dabei haben verschiedene Faktoren eine Rolle gespielt, die ich im nächsten Abschnitt „Herausforderungen“ näher erläutere.

Hauptgrund für die schlechteren Ergebnisse ist die komplexe Transformation des Input Bilds in den latenten Raum des StyleGANs. Das Problem ist, dass der Encoder das Gesicht nicht genau rekonstruieren kann und man nur eine Approximierung des Gesichts hat. Dafür wurde die Latent Optimierung eingesetzt, die durch den MSE-Loss jedoch häufig zu Artefakten und verschwommenen Gesichtern führt. [8]

Die daraus resultierenden latenten Vektoren sind unbrauchbar für die Alterstransformation, da die Artefakte ebenfalls in den manipulierten/gealterten Gesichtern auftreten.

5 Herausforderungen

Im Laufe des Projektes sind einige Schwierigkeiten aufgetreten. Zum Anfang habe ich lediglich mit der Latent Optimierung und ohne Encoder Architektur gearbeitet. Die Ergebnisse waren ausbaufähig, da häufig Artefakte im Bild aufgetreten sind. Dadurch war das Gesicht verzerrt und konnte nicht für die Alterstransformation im weiteren Verlauf genutzt werden. Die Bildqualität der generierten Gesichter war allgemein ein limitierender Faktor, da nur hochauflösende Gesichter darauffolgend manipuliert werden konnten. Unscharfe und verzerrte Bilder sind auch nach der Alterstransformation unscharf gewesen, sodass leider kein gutes und realistsches Ergebnis entstanden ist.

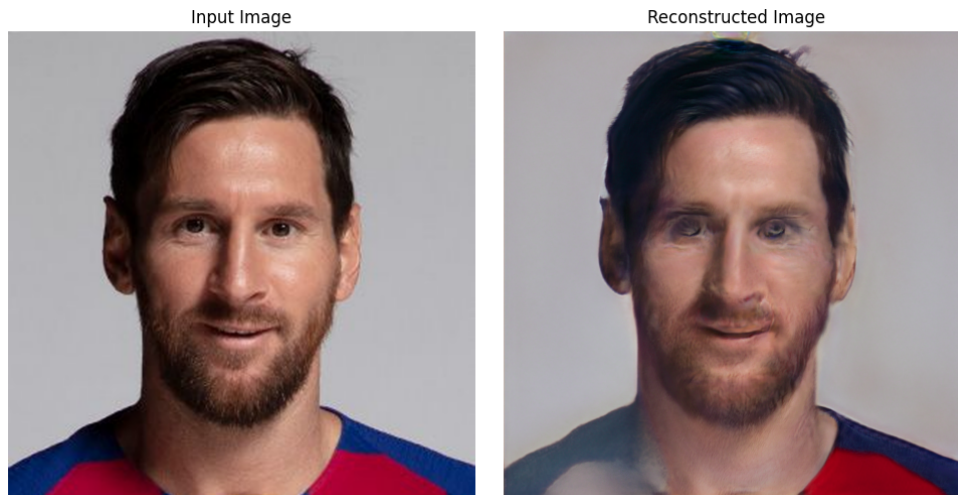


Abbildung 2 Unzureichende Bildqualität nach Latent Optimierung

Auch nach der Implementierung des Encoders sind die aus den latenten Vektoren rekonstruierten Gesichter nicht nah genug an die Input-Bilder gekommen. Die Alterstransformation mit Input-Bildern ist also nur in einigen wenigen Fällen möglich. Das Bottleneck des Encoders war der generierte Trainingsdatensatz. Die generierten Gesichter, mit ihren dazugehörigen latenten Vektoren, waren stets in einer ähnlichen Position mit nahezu identischen Verhältnissen. Das liegt an der Generierung von Gesichtern mit der StyleGAN-Architektur. Wenn nun Input-Bilder mit anderer Positionierung, anderen Lichtverhältnissen oder weiteren Merkmalen zu einem latenten Vektor umgewandelt wurde, hatte der Encoder Schwierigkeiten eine sinnvolle latente Repräsentation zu finden.

6 Next Steps

Um die gesamte Pipeline zu optimieren würde ich folgende Schritte unternehmen. Zum einen muss die Encoder Architektur verbessert werden, um bessere Annäherungen an das Input-Bild zu bekommen. Dabei könnte man einen größeren Datensatz erstellen und für mehr Variation sorgen, sodass der Encoder auch mit variierenden Inputs zurechtkommt und sinnvolle latente Vektoren ausgibt. Außerdem wäre ein nächster Schritt, den Optimierungsprozess dieses latenten Vektors zu verbessern, sodass es nicht mehr zu unscharfen Bildern und Artefakten kommt. Ist das sichergestellt, wird die Alterstransformation auch mit realen Input-Bildern sehr realistische Ergebnisse ausgeben.

- Latent-Optimierung verbessern
 - Nutzung adversarieller Trainingsmethoden, z. B. durch Einbindung eines zusätzlichen Diskriminators, um die Qualität der rekonstruierten Gesichter weiter zu verbessern.
- Datensatz erweitern

-
- Größeren Datensatz erstellen und für mehr Variation sorgen, sodass der Encoder auch mit variierenden Inputs zurechtkommt und sinnvolle latente Vektoren ausgibt.
 - Evaluation der Alterstransformation
 - Objektive Metriken, wie FID Score, um den Fortschritt zu messen.
 - Benutzerstudie mit Personen, die die Alterstransformation subjektiv mit ihrem Gesicht bewerten.

Anhang

Literaturverzeichnis

Literatur

- [1] This Person Does Not Exist. *This Person Does Not Exist*. Zugriff am 05. Februar 2025. 2025. URL: <https://this-person-does-not-exist.com>.
- [2] Ian J. Goodfellow u. a. „Generative Adversarial Networks“. In: *arXiv preprint* 1406.2661 (2014). DOI: 10.48550/arXiv.1406.2661. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [3] Tero Karras, Samuli Laine und Timo Aila. „A Style-Based Generator Architecture for Generative Adversarial Networks“. In: *arXiv preprint* arXiv:1812.04948 (2018). URL: <https://arxiv.org/abs/1812.04948>.
- [4] Yujun Shen u. a. „Interpreting the Latent Space of GANs for Semantic Face Editing“. In: *arXiv preprint* (2019). arXiv: 1907.10786. URL: <https://arxiv.org/abs/1907.10786>.
- [5] Jun-Yan Zhu u. a. „Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks“. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, S. 2223–2232. DOI: 10.1109/ICCV.2017.244.
- [6] Weihao Xia u. a. „GAN Inversion: A Survey“. In: *arXiv preprint* arXiv:2101.05278 (2021). URL: <https://arxiv.org/abs/2101.05278>.
- [7] David Bau u. a. „Seeing What a GAN Cannot Generate“. In: *arXiv preprint* arXiv:1910.11626 (2019). URL: <https://arxiv.org/abs/1910.11626>.
- [8] Grigory Antipov, Moez Baccouche und Jean-Luc Dugelay. „Face Aging with Conditional Generative Adversarial Networks“. In: *arXiv preprint* (2017). arXiv: 1702.01983. URL: <https://arxiv.org/pdf/1702.01983>.

Abbildungen

Image to Latent Vector	5
Gesicht nach Latent Optimierung	12